```
import heapq
import time


# ------------------------------------------------
# Graph: Node -> [(Neighbour, Cost)]
# ------------------------------------------------

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('E', 1), ('F', 5)],
    'D': [('G', 3)],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}


# ------------------------------------------------
# Heuristic values: estimated distance to Goal G
# ------------------------------------------------

heuristic = {
    'A': 6,
    'B': 5,
    'C': 3,
    'D': 3,
    'E': 2,
    'F': 2,
    'G': 0
```

```
}



# ------------------------------------------------
# Greedy Best First Search
# f(n) = h(n)
# ------------------------------------------------

def greedy_best_first_search(graph, heuristic, start, goal):

    priority_queue = []

    # (heuristic value, node, path, cost)
    heapq.heappush(
        priority_queue,
        (heuristic[start], start, [start], 0)
    )

    visited = set()
    nodes_explored = 0

    while priority_queue:

        h_value, node, path, cost = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)
        nodes_explored += 1
```

```
        # Goal test
        if node == goal:
            return path, cost, nodes_explored

        # Expand neighbours
        for neighbour, edge_cost in graph[node]:

            if neighbour not in visited:

                new_path = path + [neighbour]
                new_cost = cost + edge_cost

                heapq.heappush(
                    priority_queue,
                    (
                        heuristic[neighbour],
                        neighbour,
                        new_path,
                        new_cost
                    )
                )

    return None, float('inf'), nodes_explored



# ------------------------------------------------
# A* Search
# f(n) = g(n) + h(n)
# ------------------------------------------------

def a_star_search(graph, heuristic, start, goal):
```

```
priority_queue = []

# (f, g, node, path)
heapq.heappush(
   priority_queue,
   (heuristic[start], 0, start, [start])
)

best_cost = {start: 0}
nodes_explored = 0

while priority_queue:

   f_value, current_cost, node, path = heapq.heappop(
      priority_queue
   )

   # Ignore outdated entries
   if current_cost > best_cost.get(node, float('inf')):
      continue

   nodes_explored += 1

   # Goal test
   if node == goal:
      return path, current_cost, nodes_explored

   # Expand neighbours
   for neighbour, edge_cost in graph[node]:
```

```
            new_cost = current_cost + edge_cost

            if new_cost < best_cost.get(
                neighbour, float('inf')
            ):

                best_cost[neighbour] = new_cost

                f = new_cost + heuristic[neighbour]

                new_path = path + [neighbour]

                heapq.heappush(
                    priority_queue,
                    (
                        f,
                        new_cost,
                        neighbour,
                        new_path
                    )
                )

    return None, float('inf'), nodes_explored


# ------------------------------------------------
# Run Greedy Best First Search
# ------------------------------------------------

start = 'A'
goal = 'G'
```

```
start_time = time.perf_counter()

greedy_path, greedy_cost, greedy_nodes = \
  greedy_best_first_search(
    graph,
    heuristic,
    start,
    goal
  )

greedy_time = time.perf_counter() - start_time



# ------------------------------------------------
# Run A* Search
# ------------------------------------------------

start_time = time.perf_counter()

astar_path, astar_cost, astar_nodes = \
  a_star_search(
    graph,
    heuristic,
    start,
    goal
  )

astar_time = time.perf_counter() - start_time
```

```
# ------------------------------------------------
# Display Results
# ------------------------------------------------

print("=" * 45)
print("GREEDY BEST FIRST SEARCH")
print("=" * 45)

print("Path:",
    " -> ".join(greedy_path))

print("Path Cost:",
    greedy_cost)

print("Nodes Explored:",
    greedy_nodes)

print("Execution Time: {:.8f} seconds"
    .format(greedy_time))



print("\n" + "=" * 45)
print("A* SEARCH")
print("=" * 45)

print("Path:",
    " -> ".join(astar_path))

print("Path Cost:",
    astar_cost)
```

```
print("Nodes Explored:",
    astar_nodes)

print("Execution Time: {:.8f} seconds"
    .format(astar_time))



# ------------------------------------------------
# Comparison
# ------------------------------------------------

print("\n" + "=" * 45)
print("COMPARISON")
print("=" * 45)

print("Greedy Path Cost :", greedy_cost)
print("A* Path Cost    :", astar_cost)

print("Greedy Nodes Explored :", greedy_nodes)
print("A* Nodes Explored    :", astar_nodes)

print("Greedy Time : {:.8f} seconds"
    .format(greedy_time))

print("A* Time    : {:.8f} seconds"
    .format(astar_time))
```